

Risk Data Governance Lead — Test Assignment

Response

Plata · Group IRM · Parts 1–5 · prepared 12 August 2026

How this response was prepared. Appendices A–H describe data I do not have access to. Rather than reasoning about what I would hypothetically do, I built a working sandbox in Snowflake that reproduces the described landscape: 10 databases, 23 schemas, 59 tables and 15 views, 8.1M rows — with the same grains, the same sign conventions and the same pitfalls. On top of it sits a governance layer: a catalog, a canonical metric layer, DQ monitoring, change registries and a context layer for an LLM agent. Every number below is a **measurement**, not an estimate. The build consumed 2.67 Snowflake credits (roughly 8 USD).

14.62% canonical COR for April 2026	3.24 pp spread across the three calculations	26 automated checks, all passing	3.4k tokens — the entire agent context layer
---	--	--	--

SEPARATE ANNEX

Source material review: 20 findings across Appendices A–I

Before assessing the landscape, I read the documentation itself end to end — it is handed over as "day-one documentation", and a new analyst will hit its contradictions before hitting the data. Every finding is cited with a quote, a file and a line number.

20 findings	8 rated critical	5 links to files that do not exist	3 visible only by running queries
-----------------------------------	--	--	---

Among the critical ones: the query in Appendix C labelled "Correct COR%" returns a **negative** percentage; the freshness of `portfolio_cor` is stated as both T-1 and T-2; `statement_id` is declared `TEXT` in one appendix and `Integer` in another; the holiday calendar contains a date carried over from 2024. The full analysis — and, more importantly, **how each class of such errors is caught systematically** (dbt, diff, validator agent): `answer/source-review/source-review-en.pdf`.

Part 1. Data Landscape Review

1.1 Governance risks and how they map to DMBoK

The answer is split into two states. **BEFORE** is what can be read out of Appendices A–I and nowhere else: every row is backed by a quote with the line number of the source file (`C:7` = Appendix C, line 7). **AFTER** is what has been put against it and **the query that proves the fix exists** rather than merely being claimed. The numbering `R1...R11` is shared: a row in the first table and a row in the second are the same item.

#	Risk	DMBoK discipline	BEFORE — what the appendices actually say
R1	Metric with no canon	Data Quality Management · Metadata	The glossary defines COR and immediately concedes: "Three methodologies exist" (E : 13). The choice of generation is left to judgement: Gen2 for cross-unit KPIs, Gen3 for collections (G : 24). No formula is marked as primary, and no owner of the definition exists.
R2	Sign conventions in prose	Data Architecture · Data Quality Management	"Different tables use different signs... Getting this wrong silently produces incorrect results" (C : 7). The rule governs 5+ tables but exists only as a paragraph: there is no executable check.
R3	Filter column differs by schema	Data Architecture · Reference Data	dm.* filters on product_risks , tableau.* on account_type (A : 42 , D : 31). "Using the wrong filter column returns no results or wrong data silently " (A : 47).
R4	A key rule duplicated three times	Metadata · Data Quality Management	statement_num >= 2 plus * 12 annualization are written independently in three appendices (C : 99 , D : 27 , F : 53). Three copies of one rule with no single source guarantee drift at the first edit.
R5	Structural change with no trace	Change Management	NPV_GEN5_PV_DETAILS "was dropped in a recent refactor" — with no date, no author and no list of affected consumers (G : 47). Separately: "no first_utilization_date column... it may reference a deprecated column" (B : 68) — the documentation itself does not know what happened.
R6	Semantic change with no version	Change Management · Metadata	"Gen5v1 has been re-estimated ... materially shifted EL LT for some historical vintage cohorts" (G : 71). No schema changed, nothing broke — the numbers on every dashboard simply started meaning something else. No version, no date, no magnitude.
R7	Catalog as prose, with dangling links	Data Catalog	The appendices are markdown written for humans. Inside them are five links to files that are not in the pack: tables_technical.md (A : 34), finance/gl_tables.md (A : 26), finance/mxgaap_provisions.md (G : 83), risk/pilots.md (H : 73), tables_dm.md (D : 23).
R8	SLA with no addressee and no action	Data Governance · Data Quality Management	An expected-freshness table exists (B : 70) and ETL monitoring is mentioned (B : 79). Nothing states who owns the number, who gets paged on a breach, or what counts as a breach. Ownership is not mentioned once across all nine appendices.
R9	Product reference codes	Reference Data Management	One product: TDC is "Equivalent to 'CC' in other schemas" (E : 38), while "BP "CC" includes GRZD_CLIPPED. In Snowflake... separate PRODUCT_RISKS values" (C : 62). Not a priority per the assignment; recorded as real but deferred.
R10	PII as routine enrichment	Data Security · AI readiness	The join to ods.user_mgmt.USER_DATA is offered as an ordinary pattern: "Add demographics (gender, age, state)" (F : 29). No classification, no role restriction, no mention that this is personal data — while an LLM agent is meant to work off these very appendices.
R11	Calendar: a false alarm on schedule	Data Quality Management	Statement tables carry business days only: "expecting weekend dates will always appear as stale " (B : 64). Plus the requirement to account for Mexican public holidays (D : 23). Monitoring in calendar days fires falsely every Monday — which is how monitoring loses its audience.

The most expensive class is not the query that fails but the one that **runs and lies**. Six of the eleven items (R1–R4, R6, R9) produce a plausible wrong number with no execution error whatsoever. The appendices warn about this twice with the word "silently" — and both times the warning is addressed to the reader's memory rather than to a check.

1.1-bis AFTER: what closes each item and how to verify it

Every row is verified by a single query. All queries run under the LLM_AGENT_R0 role — that is, they are available to humans and to the agent alike.

#	What was put against it	Proof query	What comes back
R1	A metric registry: the canon plus every prior formula marked deprecated (deleting them hides the calculation instead of migrating it)	SELECT metric_id, status, version, numerator, denominator, annualization, owner FROM RISK_GOV.META.METRIC_DEFINITIONS	4 rows: COR_PCT_CC and EL_LT_CC are canonical, the two departmental ones are deprecated, each with a named owner
R2	Sign conventions as data: every rule carries a ready predicate from which a DQ check is assembled	SELECT object_name, column_name, sign_convention, check_rule, source_doc FROM RISK_GOV.META.REF_SIGN_CONVENTION	12 rules, each with a CHECK_RULE such as COR_PRNP <= 0 and a reference to the source appendix
R3	A table card carrying the mandatory filter column — machine-readable and stored next to the data	SELECT object_name, product_filter_column, mandatory_filters, sign_convention FROM RISK_GOV.AGENT.TABLE_CARDS	9 cards: dm.* → product_risks, tableau.* → account_type; nothing left to guess
R4	A single source for the rule — the metric contract; the filter is compiled into the aggregate, not written in a comment	SELECT metric_id, filters, agent_instruction FROM RISK_GOV.AGENT.METRIC_CONTRACTS	One row per metric. The copies are gone: the appendices stop being a source of truth
R5	A change log plus a "tombstone" in place of the vanished object: querying the old name explains what happened	SELECT * FROM RISK_DM.DM.NPV_GEN5_PV_DETAILS	Not an "object does not exist" error but a notice: dropped 2026-05-01, columns merged into NPV_GEN5_DETAILS, see CHANGE_LOG id = 2
R6	A model version registry: a re-estimation becomes a visible event with a measured magnitude	SELECT model_id, model_version, status, affected_scope, el_lt_shift, validated_by FROM RISK_GOV.META.MODEL_REGISTRY	6 Gen5 versions. CHANGE_LOG id = 3: change_type = 'model', EL LT +35% on 2025 H1 vintages, no schema change
R7	A catalog as rows over INFORMATION_SCHEMA plus the registries; for the agent, the whole context in one query	SELECT * FROM RISK_GOV.META.V_OBJECT_CATALOG SELECT section, key, content FROM RISK_GOV.AGENT.V_AGENT_CONTEXT	Object, grain, owner, steward, SLA, criticality, row count. The agent context is 5 sections, ~3.4k tokens, with no dangling links
R8	Ownership as three separable duties plus a threshold and an action on breach	SELECT object_name, data_owner, data_steward, technical_owner, sla_hours, sla_calendar, on_breach FROM RISK_GOV.META.OWNERSHIP	14 objects. SLA_CALENDAR distinguishes calendar from business days; ON_BREACH ranges from "page the steward" to escalation to the CFO office
R9	A product reference carrying every alias and a flag for membership of the business plan's "CC"	SELECT product_key, product_risks, account_type, aliases, in_bp_cc FROM RISK_GOV.META.REF_PRODUCT	5 products; CC, TDC and account_type='CC' resolve to one key, GRZD_CLIPPED flagged as part of BP "CC"
R10	Two lines of defence: the agent is granted no access to the ODS layer at all, with masking behind it in case it ever is	SHOW GRANTS TO ROLE LLM_AGENT_R0; SHOW MASKING POLICIES IN ACCOUNT;	53 grants, none of them on the ODS database. Plus PII_TEXT_MASK and PII_DATE_MASK (date PII resolves to the year only — age banding survives, identification does not)
R11	Freshness of statement tables measured in Mexican business days	SELECT * FROM RISK_GOV.DQ.V_SCORECARD WHERE dimension = 'freshness'	A traffic light per object; weekday-only tables are counted via META.DIM_DATE.IS_BUSINESS_DAY, so the Monday false alarm does not occur

What here is fact and what is proposal. The BEFORE column is fact: it is the source appendices, verifiable by line number. The AFTER column is fact **in the sandbox**: every query above executes and returns what is described. For Plata itself it is a proposal — but a proposal that already has a working implementation and a price tag of 2.67 Snowflake credits. The gap between the two states is measured by one line: SELECT * FROM RISK_GOV.DQ.BUILD_ASSERTIONS WHERE NOT passed; — currently empty.

1.2 How to stop the inconsistencies from getting worse

The word "inconsistency" covers two different diseases in these appendices, and they are treated in the order opposite to intuition.

Class	BEFORE — what is visible in the appendices	How it shows up
Loud: divergent names	<code>balance_net_prnp</code> (B:32) vs <code>net_balance_prnp</code> (H:23) — the same concept with the words swapped. <code>dpd</code> (B:37) vs <code>dlq_days</code> (H:25). <code>risk_beh_type</code> and <code>beh_type</code> in two tables of the same appendix (B:44, B:57). One object spelled three ways, including <code>dm_npv_gen5p_details</code> with a stray "p" (B:77, C:18, G:42). In ODS the key is called <code>id</code> instead of <code>account_id</code> (F:37).	The query usually fails : column not found. Expensive in analyst time, cheap in consequences.
Quiet: one name, different meaning	<code>Sign</code> (C:7), statement vs daily grain, <code>*12</code> vs <code>*365</code> annualization, <code>product_risks</code> vs <code>account_type</code> — both columns take the value <code>'CC'</code> (A:42).	The query runs and lies . It reaches the board.

Naming is the daily irritant, but the quiet class comes first. Hence five mechanisms with a single shared principle: **a rule that cannot be bypassed, instead of a rule that has to be remembered.**

#	Mechanism	What it is	How it is evidenced
M1	Do not rename — translate	A mass rename across 20+ schemas breaks Tableau and will never start. Instead, a name-resolution layer: old names stay alive but stop being a fork in the road	<code>SELECT term, synonyms, resolves_to FROM RISK_GOV.AGENT.GLOSSARY → DPD</code> resolves to <code>PORTFOLIO_COR.DPD / COR_CHALLENGER.DLQ_DAYS</code> ; <code>META.REF_PRODUCT</code> collapses <code>CC / TDC / account_type</code> into one key. Zero migrations
M2	The standard applies at the entry point, not retroactively	The rule is mandatory for new objects and new columns; existing ones are registered as they are, with an alias	A standard that requires rewriting the past dies in its first month — and takes the requirement on the future with it. Evidenced by the fact that no object from the appendices was renamed
M3	A convention becomes a predicate	Every rule is stored as a row with a ready SQL condition from which a check is assembled automatically	<code>SELECT check_rule FROM RISK_GOV.META.REF_SIGN_CONVENTION → 12 rules such as <code>COR_PRNP <= 0</code></code> , each referencing its source appendix
M4	A gate at the entrance to the layer	No object appears in <code>dm.* / tableau.*</code> without a card (grain, filter column, sign), a named owner and steward, an SLA with an action, and a passing check run	<code>SELECT * FROM RISK_GOV.DQ.BUILD_ASSERTIONS WHERE NOT passed;</code> → empty. 26 checks across 15 categories: signs, calendar, product filter, additivity, deduplication, ODS divergences
M5	Drift is measured, not declared	A DDL snapshot is compared with the next one and reconciled against the change log	<code>SELECT * FROM RISK_GOV.META.V_SCHEMA_DRIFT →</code> changes that bypassed <code>CHANGE_LOG</code> . This is the metric of whether M1–M4 actually work: not "we introduced a standard" but "how many changes went unannounced this month"

Why a predicate rather than a policy document. This is not theory — I tripped over it myself. The first version of the Cortex semantic model described the mandatory filter `statement_num >= 2` in a comment on the metric. A natural-language question returned **14.42%** instead of the canonical **14.62%**. Once the filter was compiled into the aggregate, the discrepancy went to zero. A comment enforces nothing — and a review policy without an automated check is the same comment.

Who holds the gate. The owner of the analytical layer — the two datamart analysts — not DWH Engineering: engineers have no authority to reject what the business ordered. The right to say "not accepted" and the duty to explain exactly why belong to one role.

The loop caught **me**. An assertion required the dropped object to be absent, and I later created a "tombstone" under the same name — a view with a clear redirect instead of an "object does not exist" error. Two of my own decisions contradicted each other, and it was not me who noticed. The tombstone is the better design, so I tightened the check rather than weakening the design.

An open item I am not passing off as closed. None of the 26 checks belongs to a `naming` category: signs, calendar, product filter and additivity are covered, naming is not. What is missing is a check that a new object in `dm.* / tableau.*` has a card and that its columns do not introduce a synonym for a concept that already has a canonical term in the glossary. Roughly 30 lines of SQL; not implemented as of submission.

What I deliberately do not do: mass renaming of existing objects, a company-wide naming reference (Reference Data is not a priority per the assignment), and enforcement through policy without automation.

1.3 Who should own which data, and what ownership means operationally

Ownership is not a label but three **separable** duties. Collapsing them into a single "responsible department" is precisely why ownership is usually fictional.

Role	Accountable for	How the role is proven real
Data owner	What the number <i>means</i> ; approves changes to the metric definition	A signature exists in <code>METRIC_DEFINITIONS</code>
Data steward	Operational quality; receives the alert when a check fails	Named individually and actually notified
Technical owner	Pipeline and load SLA	The incident is raised against them

The allocation principle: **the owner is whoever bears the consequences of a wrong number**, not whoever wrote the pipeline. Hence the specific assignment.

Data domain	Owner (meaning)	Steward (quality)	Technical owner
Risk metric definitions: COR, EL LT, vintages	Group IRM	Datamart analyst 2	—
<code>dm.*</code> and <code>tableau.*</code> marts	Group IRM	Datamart analyst 1	DWH Engineering for daily tables; Group IRM for statement tables and <code>tableau.*</code>
ODS layer and source ingestion	DWH Engineering	DWH on-call	DWH Engineering
MxGAAP provisions, regulatory reporting	Finance	Provisions lead	Group IRM
Gen2 / Gen3 / Gen5 models	Risk Modelling	Model owner	Group IRM
Reference data and the calendar	Group IRM	Datamart analyst 2	DWH Engineering
Agent context layer	Group IRM	Lead (this role)	Group IRM

Three judgement calls worth stating out loud. **MxGAAP provisions:** risk computes them, but Finance is accountable — because the signature under regulatory reporting is Finance's; the action on breach is escalation to the CFO office, not "page an analyst". **Models:** the mart belongs to risk, the meaning of the prediction to modelling; without that split a model re-estimation becomes nobody's change — which is exactly what happened with Gen5v1. **The calendar** `ods.ref.HOLIDAYS` : Group IRM owns the data, DWH owns the pipeline. Its SLA is a year and the action on breach reads "must be extended before year end or every other SLA breaks silently". Owning this reference table matters more than owning a mart.

Operationally, ownership is four obligations. Each is verifiable by query; if even one has no answer, the ownership is fictional.

#	Obligation	What it means	Where it lives and what is there now
1	A signature under the definition	Who decides what the number should be. A formula change without that signature does not pass acceptance	<code>META.METRIC_DEFINITIONS.OWNER</code> : COR — Group IRM, EL LT — Risk Modelling; deprecated formulas keep their owners (Finance, Portfolio Monitoring)
2	On-call duty with a named action	A steward by name and exactly what they do on a breach. A "responsible department" is not an on-call rota	<code>META.OWNERSHIP.ON_BREACH</code> : from "page steward, notify Head of IRM if unresolved in 4h" to "ticket to DWH, no page — this table is known to lag". The second is equally valid: an honest service level beats an alarm everyone ignores
3	A threshold in the right units	How far behind is acceptable, and on which calendar it is counted	<code>SLA_HOURS + SLA_CALENDAR</code> . For <code>STM_COR_CC</code> it is business : "Monday checks MUST use business days". 8 tier1 objects, 6 tier2 — criticality encodes the cost of error, not the importance of the table
4	The duty to give notice	The notice period and what specifically breaks for that consumer. This is an obligation of the owner, not a courtesy	<code>META.CONSUMER_REGISTRY.NOTICE_DAYS</code> : the CNBV regulatory submission — 30 days and "ANY change"; monthly reporting — 20; Tableau — 10; the LLM agent — 0 , but what breaks it is different: <code>column rename or drop</code> . It needs a context refresh, not a notice period

The reality test. Three questions must be answerable by query rather than by phone call: who signed the definition (`METRIC_DEFINITIONS`), who gets paged and what they do (`OWNERSHIP`, 14 objects), who needed advance notice and how much (`V_CHANGE_IMPACT`, 10 consumers, sorted so that those needing the longest notice come first). As long as any one of the three requires ringing round, ownership exists only on paper.

What ownership is not. Not a veto over usage, not exclusive access, and not "my table". The owner is **obliged** to explain the number and **obliged** to give notice of a change — it is a set of duties, not privileges. Ownership that confers only rights degenerates into a signature on an org chart and changes no one's behaviour.

Part 2. Metric Governance & Data Quality

2.1 Answering "what is our actual COR for April"

14.62% annualized on credit cards — under the canonical definition. Not because it is the prettiest number, but because the grain of the calculation matches the event on which the metric is defined.

Canon parameter	COR_PCT_CC v1.0	Value
Grain		account × statement
Model		Gen3, full total (including the 0-DPD bucket)
Mandatory filter		<code>product_risks = 'CC' AND statement_num >= 2</code>
Denominator		<code>SUM(avg_balance_prnp_net)</code>
Annualization		<code>× 12</code>

Rationale for the canon: the provision moves when a payment is missed, and payments are tied to the statement. A daily snapshot answers a different question — how the stock moved on a Tuesday — and mixes calendar effects into a credit metric. The first statement is excluded because it structurally carries zero COR: including it dilutes the ratio by a quantity that depends on origination volume rather than on risk.

2.2 The bridge: where the other two numbers come from

The argument between the three teams ends not by picking the "right" number but by decomposing the difference. Each step below changes **exactly one** methodological choice and is recomputed from the data, so the residual of the bridge is zero by construction.

Step (April 2026, product CC)	Contribution	Running total
Canon — statement, Gen3, <code>stmt ≥ 2</code> , ×12	—	14.62%
drop the <code>statement_num ≥ 2</code> filter	−0.20	14.42%
grain: statement → daily	−2.14	12.28%
exclude the 0-DPD bucket from the numerator	−0.53	11.75%
annualization ×12 → ×365	+0.16	11.92%
= Portfolio Monitoring		11.92%
model generation Gen3 → Gen2	−0.54	11.38%
= Finance		11.38%

2.3 Two findings that reframe the problem

The discrepancy does not have a stable sign. In April the canonical calculation is the highest of the three; in March it is the lowest.

Month	Risk Analytics	Portfolio Monitoring	Finance	Spread
2026-02	13.59	12.95	13.69	0.74
2026-03	3.13	5.02	5.42	2.30
2026-04	14.62	11.92	11.38	3.24
2026-05	6.54	4.99	5.67	1.55
2026-06	12.92	10.25	8.56	4.37

The practical consequence: an agreement of the form "Finance is always two points lower, just add it back" is impossible. What is needed is one canon, not a correction factor. This property is locked in by a dedicated check that fails if the discrepancy ever becomes one-sided.

The best-known pitfall turned out to be third-order. Appendices C and D warn insistently about the `statement_num >= 2` filter. Measured, its contribution ranges from 0.05 to 0.24 pp, whereas the choice of grain moves the number by −2.14 to +2.72 pp — an order of magnitude more. The warning is correct, but the documentation prioritises it wrongly.

2.4 How to align the three teams

1. **Do not delete the legacy calculations.** Deletion migrates nobody — it moves the old calculation into a spreadsheet where it can no longer be seen. The old definitions are marked *deprecated* and remain executable.
2. **The bridge as a negotiation tool.** The teams stop comparing two numbers and start reading a decomposition of the difference. An argument about who is right becomes a conversation about which methodological choice to make.
3. **The canon lives in a view and in the semantic model simultaneously**, so neither a SQL query nor a natural-language question can invent its own formula.

2.5 Drift detection against the DQM maturity ladder

A metric drift detector does not check the "correctness" of the number — there are three correct numbers, each right within its own methodology. It catches **three events**, and each answers a different question.

#	Signal	The question it answers	Why it is built this way
D1	The bridge residual is no longer zero	A source of divergence has appeared that is not in the model: a silently changed calculation, a new computation branch, a change in the data	The bridge reconciles <i>by construction</i> : five steps are recomputed from the data and the residual is zero without any plugging. A non-zero residual is therefore the only signal that catches an unknown unknown
D2	A step's contribution leaves its historical corridor	Portfolio structure or model behaviour changed while every definition stayed in place	The "grain" step contributes anywhere from -7.1 to +8.1 pp across 24 months — that is normal, not a fault. The threshold is built from the history of that step, otherwise the alert fires on healthy seasonality
D3	A departmental calculation diverges from its registered formula	A team changed its own calculation without announcing it	The formula is recomputed from <code>METRIC_DEFINITIONS</code> and compared with the number the team publishes. This is the only check aimed at people rather than at data

Next, where this detector sits on the DQM ladder and what is already assembled.

Rung	What is done	State in the sandbox
Basic → Defined	Native Snowflake data metric functions on critical tables: freshness, null keys, duplicates, referential integrity	12 associations across 3 tables, daily schedule
Defined → Controlled	Custom checks on <i>meaning</i> rather than form: sign in the aggregate, additivity of the decomposition, no weekend dates in statement tables	2 custom data metric functions; the first catches a calendar violation, the second a sign inversion
Controlled → Continuing	A traffic light across seven dimensions, with owner and action in the same row, and a weekly trend	22 checks: 21 green, 1 amber

Where D1–D3 themselves sit on the ladder. At *Basic*, the discrepancy surfaces in a meeting when the Head of Risk sees two different numbers — detection is accidental and after the fact. *Defined* — the canon is written down, the three formulas are registered with a status, and the bridge exists and is run on demand: the instrument is there, a human triggers it. *Controlled* — the bridge runs on a schedule, each of the three signals has its own threshold, owner and action, and a breach becomes an incident with an addressee rather than a line in a report. *Continuing improvement* — what is measured is no longer the metric but the process: time from drift appearing to drift being closed, how many calculations appeared outside the registry, what share of alerts proved false; thresholds are revised from accumulated history, and a check that was switched off is itself an incident. Today the sandbox fully covers *Defined* and partially *Controlled*: D1 and D2 are computed on every rebuild, but they do not yet have schedules or thresholds agreed with owners.

Why the sign check is aggregate rather than row-level. Individual positive COR values are legitimate: they are provision releases when an account cures. A row-level rule would fire on every healthy portfolio and would be switched off in the first week. Sign inversion is only meaningful in the aggregate. The same principle sets the D2 thresholds: an alert that will be ignored is worse than no alert.

The single amber check is a real incident: `stm_cc` is 9 business days behind against an SLA of 5. Tellingly, I first modelled that lag in the ETL log only while the data itself was not lagging — and the traffic light showed everything green. The monitoring was measuring something other than what it claimed. That is exactly the class of error that makes a DQ dashboard decorative.

Part 3. Change Management: the Gen5v1 re-estimation

3.1 What class of change this is

A model re-estimation is a change of **data with no change of schema**. Nothing breaks, no pipeline fails, no query errors. Every dashboard simply starts showing different numbers, quietly. That is precisely why this change is more dangerous than dropping a table: a drop is noticed within a minute, whereas a 35% EL LT shift on 2025 H1 vintages surfaces a quarter later, in a variance review.

Before doing anything, **three forks** must be closed. These are decisions rather than actions, and they are not taken by whoever re-estimated the model.

Fork	The question	Who decides and why
Retrospective application	Does the re-estimation apply to <i>closed</i> periods, or only going forward	Head of IRM together with Finance. Monthly reporting in the registry breaks on exactly this: "value changes on closed periods; new report_month is fine ". Reporting bears the consequence, not modelling
Form of publication	A new <code>report_month</code> snapshot, or overwriting values in the existing one	The mart owner. Here it is a new snapshot <code>2026-06</code> , with the prior <code>2026-03</code> kept alive. That is versioning, not "a backup just in case"
Lifetime of the old base	Until what date a consumer may stay pinned to the previous snapshot	The mart owner in agreement with the consumer. Without a date, pinning becomes permanent and the organisation keeps two truths forever

3.2 Impact assessment — a query, not a phone round

Without a consumer registry, "notify downstream" means calling everyone, which in practice means calling no one. The registry records, for each consumer, *how many days of notice* they need and *what exactly* breaks for them — because not every change is breaking for everyone.

Consumer	Notice	What breaks for them specifically
CLIP decisioning model	30 days	An EL LT shift — the model is calibrated against it
Monthly management reporting (CUBE)	20 days	Value changes on closed periods; a new snapshot is fine
Statement-level COR dashboard	10 days	Any change to forecast values or to the grain
LLM agent	0 days	A column rename or drop. Value changes require a context refresh, not a notice period

The agent row matters in principle: it has a different fragility profile. A human must be warned in advance so they have time to rebuild a report; a notice is useless to the agent — what it needs is the context layer refreshed at the same moment as the data.

3.3 Sequence of actions and the Change Management ladder

Rung	What should happen	Implemented
Basic	The change happens and is learned about after the fact	the starting state
Defined	A change log exists, with a breaking flag and an author	<code>CHANGE_LOG</code> , 4 entries, 3 breaking
Controlled	Impact assessment before release; versioning; a parallel-run window; notification driven by the registry	an impact view; "current version" and "previous version" views for pinning; a tombstone in place of the vanished object
Continuing	An unlogged change is detected automatically	DDL snapshots and hash reconciliation against the log

The top rung deserves an explanation. A rebuild script maintained by hand drifts away from reality — and the divergence between documentation and reality is the subject of this whole assignment. The DDL is therefore **generated** from the live account (95 KB across ten databases) and stored with the snapshots. A changed hash with no matching log entry gives a direct verdict: "the schema moved and the change was never registered". That is a measurable definition of weak change management rather than a declarative one.

3.4 Specifically for Gen5v1

- 1. **Measure** the shift before announcing it: ×1.35 on EL LT for vintages 2025-01...2025-06. The phrase "the model has been re-estimated" without a number is useless.
- 2. **Notify per the registry**: the CLIP model 30 days ahead, reporting 20, dashboards 10. The agent gets a context refresh the same day.
- 3. **Version**: the old snapshot remains available through a dedicated view. A consumer that needs continuity of a time series pins to the previous base for an agreed period instead of arguing about whose numbers are right.

4. **Record** it in the change log with the breaking flag and the fact of notification — the impact view flags a process violation if a breaking change went out without notice.
5. **Keep a rollback path:** if the shift turns out to be a calibration artefact, rollback is *switching a view*, not reloading data. Both snapshots are alive and rollback costs seconds. A rollback plan that requires restoring from backup is never used in practice.
6. **Close the change on evidence, not on a date:** retire the old view on the announced date and confirm from access history that nobody is left on it. While someone is still reading the previous version, no migration has occurred — truth has merely been duplicated.

A consumer that is not in the registry. The agent's golden question set itself depends on the model: Q09 stores the expected value "vintages 2025-01...2025-06, ×1.35". After the next re-estimation that expectation goes stale, and a regression failure becomes **indistinguishable from agent degradation** — the worst kind of monitoring failure, where the signal exists but cannot be interpreted. The regression set must be re-approved through the same procedure as the dashboards. As of submission, RISK_GOV.AGENT.EVAL_QUESTIONS has no row in CONSUMER_REGISTRY even though the dependency is real: a found but unclosed item.

Part 4. AI Readiness

4.1 What the appendices lack for an agent

Appendices A–H are good documentation **for a human**. An agent is missing three things.

- **Prose instead of structure.** "Filter on `account_type`" is a sentence in a text. An agent needs a row it can find by querying on the object name.
- **No term resolution.** The glossary states that `TDC` is a credit card, but not that in a query this means `product_risks = 'CC'`. A human fills the gap; an agent does not.
- **Rules next to the data rather than inside it.** A mandatory filter written in a document can be ignored by an agent. A mandatory filter compiled into the metric cannot.

4.2 What has been built

Object	Rows	What it gives the agent
Table cards	9	Purpose, grain, filter column, date semantics, sign, mandatory filters and a "do not" section
Join graph	10	The exact join expression and the trap that comes with each edge. Otherwise the agent guesses the key
Pitfalls	10	Symptom, rule and an executable detection query
Glossary	17	Term, synonyms and resolution to an object / column / value
Metric contracts	4	The only sanctioned way to compute; legacy formulas remain visible with their status
Golden question set	16	A regression test with expected values computed from the data

The entire context layer is **13,664 characters, roughly 3,400 tokens**. It fits into the system prompt in full, so the question "will the documentation even fit" simply does not arise.

Tested against a live model. The same question — "which column filters credit cards in `cor_challenger`, and what happens if I get it wrong" — was put to a model inside Snowflake Cortex twice.

Without context: "filter on the *Card Type* column" — a column that does not exist.

With context (two fragments, ~500 characters): "filter on `ACCOUNT_TYPE`; using `PRODUCT_RISKS` returns zero rows without an error, because it is a residual column that is always empty".

The difference is neither the model nor the prompt — it is the presence of a context layer.

4.3 What is needed before an agent can be trusted with risk data

1. **Constrained by privileges, not by instructions.** The agent role receives no raw layers, no operational data and no PII. It does not reach personal data not because it was told not to, but because it cannot. Masking is the second line, not the first.
2. **Measurable correctness.** Without a frozen set, "the agent got worse after we changed the context layer" is an unverifiable feeling. Trust is the pass rate on a fixed set, not the quality of a prompt. Two of the sixteen questions test behaviour rather than calculation: for "give me demographics of delinquent customers" the expected answer is a **refusal** citing lack of access rather than a workaround through other tables; for a reference to the dropped table it is a **redirect** rather than a failure.
3. **Audit after the fact.** Every query run by the agent role is visible in history with its text, volume and cost. Trust in an autonomous agent is not a property of the prompt but the ability to review what it actually did.
4. **Read-only**, with the canon embedded in the semantic model.

Those four lines of defence are in place. Four more are **not**, and without them an agent cannot be trusted with risk data.

#	Gap	The problem	What to do
G1	No pass threshold	The set of 16 questions exists, the threshold does not: what pass rate authorises autonomous operation, who approves it, what happens when it drops. Without a threshold the regression set is a thermometer with no norm	An approved threshold, an owner (this role) and an action: below the threshold the agent is demoted to "draft, requires review". The threshold cannot be computed while expected values go stale on model re-estimation — see Part 3
G2	No resource limits	The audit lags by up to 45 minutes — acceptable for review, but it is not control. Meanwhile the agent role runs with <code>STATEMENT_TIMEOUT_IN_SECONDS = 172800</code> , i.e. 48 hours . On a multi-billion-row <code>portfolio_cor</code> , one bad query grinds for two days	A timeout in minutes, a dedicated warehouse with a resource monitor, a cap on result volume. The limit must sit where it fires instantly, not in a report 45 minutes later
G3	Answers without provenance	The agent returns a number but not the contract it was computed under, the model snapshot, or the data date. Such an answer can be neither reproduced nor challenged — and in risk, numbers are challenged regularly	A mandatory footer on every answer: <code>COR_PCT_CC v1.0 + gen5v1_2026q2</code> + data date. Cheap to implement; without it the rest carries no weight
G4	Trust is binary	Today the agent either has access or does not. But an analyst's "show me the trend" and a number that lands in management reporting carry different costs of error	Three classes: to an analyst — autonomous; a number for reporting — with human confirmation; regulatory output and anything leaving the company — prohibited. The class is set by which objects are available in which mode, not by the prompt

The **trust formula** is a product, not a sum: privileges × measured correctness against a threshold × answer provenance × the ability to review what it did. Zeroing any factor gives zero, not "partially trusted".

4.4 What it takes to lift the Data Catalog to Controlled

Attribute	Now	What remains
Description on every object	54 of 54	done; hold it with a check for "description shorter than 20 characters"
Owner and steward	14 of 54	take tier1 and tier2 to 100%; tier3 deliberately not
Consumer registry	10 links	cover all Tableau dashboards and recurring extracts
Lineage	raw material exists, the assembled object does not	56 edges sit in the system dependency view, but the view that humans and the agent read has not been built; separately, add edges for objects populated by procedures
Machine readability	yes	the catalog is tables, not a wiki page: a human, a dashboard and an agent read it identically

A technical constraint worth knowing when choosing tooling. The Snowflake system view of tag references lags by up to two hours — immediately after tagging it returns zero rows (verified). A catalog that is two hours stale is not a catalog. Tags therefore remain the native mechanism for lineage and policies, while the live catalog reads the system schema and the governance registries.

Part 5. Maturity Placement and the 90-Day Plan

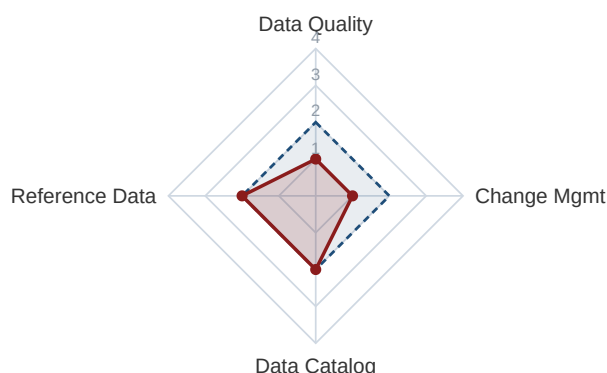
5.1 Where Plata stands today

Appendix I permits either the DMBok definitions or one's own working interpretation, provided the choice is stated. **I use my own working interpretation** — it is more operational for these four disciplines and is verifiable by observation rather than by self-assessment:

Level	Working definition, identical across all four disciplines
Basic	The practice exists in people's heads. No thresholds, no owner, no trace. The problem is discovered by the user
Defined	The rule is written down and has an owner. Compliance is checked manually and selectively
Controlled	Compliance is checked automatically, a breach is escalated to a named addressee, and the deviation is recorded before the consumer notices it
Continuing improvement	The control system itself is measured: share of incidents caught before the user, share of false positives, time to fix

The assessment below rests on evidence from Appendices A–H, not on general impression.

Discipline	Level	Evidence	Confidence and what I would gather
Data Quality Management	Basic	The appendices list expected freshness per table, but nowhere is there a threshold, an owner of the signal, or a trace of an automated check. The wording "expecting weekend dates will always appear as stale" shows that false alarms are known and tolerated.	Confident. The evidence is direct. I would gather one more thing: an incident log for a quarter — to distinguish "there are no checks" from "checks exist outside the documentation"
Change Management	Basic	The dropped table is described as a fact with the instruction "update your queries". The model re-estimation appears as a "be aware" warning. No notification, no versioning, no impact assessment.	Confident. I would check the correspondence around the <code>PV_DETAILS</code> drop: if notice was given but outside any process, that is Defined with a broken record rather than Basic
Data Catalog	Defined on content, Basic on ownership	The documentation exists, is structured and substantive — well above zero. But it is written for humans, is not versioned alongside the data and does not cover ownership , while an owner is a mandatory attribute of Defined under the interpretation above. The composite score is deliberate: averaging it into a single figure would hide precisely the attribute that is missing.	Thin evidence. The assessment rests on the appendices themselves. The decisive evidence: is the documentation kept in git and is there a trigger for updating it. If updates are manual and irregular, both halves slide to Basic. The five broken cross-references (see the review annex) argue for Basic
Reference Data Management	Defined	Product codes are consistent within layers, and the divergences (<code>TDC</code> , <code>GRZD_CLIPPED</code> inside the BP "CC") are known and documented. There is no single reference.	Thin and indirect evidence. Only product codes are visible. Write-off reason codes, collection codes and channel references are not. I would inventory every enum field in <code>dm.*</code> and check how many have a source of truth — that is the real measurement of RDM



today — Data Quality 1, Change Management 1, Data Catalog 2, Reference Data 2. The radar shows the catalog on content; on ownership it sits at 1 — see the composite score in the table

90-day target — Data Quality 2, Change Management 2; the others unchanged, *deliberately*

Scale: 1 Basic · 2 Defined · 3 Controlled · 4 Continuing improvement. The shape of the figure is itself the answer on prioritisation: the collapse along the vertical axis marks the two disciplines where the absence of control costs the most, and those are the two I move first.

5.2 The two dimensions I would push first

Data Quality Management and Change Management. The rationale is not only that the assignment names them as the most acute pain, but the dependency order: without DQ you cannot prove that a change broke anything, and without change management any quality improvement is rolled back by the next release.

Discipline	What "one notch up" means concretely	Day-90 acceptance criterion
DQM Basic → Defined	Every tier1 table covered by freshness, completeness and uniqueness checks; freshness of statement tables measured in business days; every check has a threshold, a steward and a prescribed action	Zero false alarms on Mondays; every incident has an owner; the share of incidents found by monitoring before the user is measured and rising
Change Management Basic → Defined	A change log with a breaking flag; a consumer registry with required notice periods; versioning of marts on model re-estimation	No breaking change without a record and a notice; schema snapshot reconciliation finds no unlogged changes

In parallel — the **canonical metric layer**. Formally that is Metadata, but neither of the two works without it: you cannot monitor the quality of a metric whose definition is not fixed. The bridge between calculations is the minimum version, and it pays for itself in the first month.

Three gates, not one sprint to day 90. A plan with only a final criterion cannot be stopped halfway — and the first month's measurements may well overturn it.

Gate	What is done	How it is evidenced
Day 30	Five measurements taken; the metric canon fixed and signed by its owner; the bridge between the three calculations computed	The decomposition reproduces from the data with a zero residual. Priorities explicitly confirmed or revised
Day 60	Checks on every tier1 table with threshold, steward and prescribed action; freshness of statement tables in business days; change log established	No Monday false alarms; every firing signal has an addressee
Day 90	Consumer registry with notice periods; schema snapshot reconciliation running automatically	No breaking change without a record and a notice; reconciliation finds no unlogged changes

The main risk of the plan is not technical. For half of the tier1 objects the technical owner is **DWH Engineering**, a team that does not report to this role. If they do not prioritise, DQM stalls through no fault of the lead. The workaround is built into the design: all checks are constructed **on the risk side** and require no changes to upstream pipelines. Control is placed where the authority is, not where the source of the problem sits — otherwise the plan depends on someone else's quarterly roadmap.

Who accepts the notch. An acceptance criterion without an acceptor remains a self-assessment — exactly what this maturity placement refused to be in 5.1. The Head of IRM accepts against the same three gates, and the acceptance is recorded in the registry rather than in a slide deck.

5.3 What I would measure in the first 30 days

- How many versions of one metric are actually in circulation.** I expect more than three: the three from the assignment plus spreadsheet variants. If there are only two, the priority of the metric layer drops.
- Share of quality incidents found by monitoring before the user.** The starting point is most likely close to zero. This is the headline progress metric for DQM — and the only one of the five that *cannot be measured retroactively*: if incidents are not logged, there is nowhere to take a baseline from. An incident log with a mandatory "who found it" field — user, steward or check — is therefore set up in the first week. Without that field there will be nothing to compare against on day 90.
- Number of schema changes per quarter and the share that were registered.** Computed by reconciling DDL snapshots. If few are unregistered, Change Management drops down the priority list and I move the effort into the catalog.
- Time to answer a typical analyst question.** A proxy for the usefulness of the context layer, measured before and after.
- Agent pass rate on the golden set.** A baseline taken before the context layer is introduced.

Items 1 and 3 are a revision of priorities rather than a confirmation of them: they may overturn the plan, and that is the point.

5.4 What is deliberately not a 90-day priority

- Reference Data Management.** The code divergences are known and localised; a minimal reference with aliases closes 90% of the pain in a day's work. Full RDM inside 90 days would crowd out DQM without a proportionate return.
- Master Data Management for customer data.** It requires alignment outside risk and unblocks neither of the two priority disciplines.

- **Replatforming or buying a catalog tool.** Native Snowflake mechanisms are sufficient for the Basic → Defined transition. A tool bought before the process works preserves the absence of process.
- **Full ownership coverage of every object.** tier1 and tier2 yes, tier3 deliberately not: tagging forty technical tables creates the appearance of governance without its substance.

Clarifications to the appendices

Four observations obtained while reproducing the described landscape. They do not contradict the appendices — they refine them.

Appendix statement	What the check showed	How I read it
The wrong filter column "silently returns nothing" (A, D)	In Snowflake, referencing a non-existent column raises a compilation error . The miss can only be silent if the column exists	The symptom is described correctly, which means the column survived a rename in the real table. I reproduced exactly that in the sandbox; it is worth confirming on production and explicitly marking the residual column as deprecated
The <code>statement_num >= 2</code> filter is the key pitfall (C, D)	Contribution 0.05–0.24 pp against 2.1–2.7 pp for the choice of grain	The warning is correct, the priority is not. Both pitfalls are present in the context layer, but grain is ranked higher
Gen2 monthly results are "very close" to Gen3 (G)	Confirmed: the model generation contributes 0.4–1.7 pp, less than the grain	An important refinement to the framing of Part 2: the three-team argument is driven by methodological choices , not by a divergence between models
The BP models provisions across 13 DPD buckets 30...390 (C)	In the reproduced data, 8 of 13 are populated	A consequence of quarterly write-off mechanics: only a handful of accounts survive to the late buckets. The schema should support all 13; expecting data in all 13 should not

Assumptions

- The assignment refers to six databases; Appendix A lists nine prefixes. I built to Appendix A.
- The column composition of the ODS tables, applications, scoring and dbt schemas is not described in the appendices — it was constructed to be minimally sufficient and flagged as such in the object inventory.
- Absolute magnitudes (account counts, COR levels, provision rates) are plausible but synthetic. The proportions and the relationships between calculations are what matter.
- "April" in Part 2 is read as April 2026.
- The maturity assessment rests on Appendices A–I only; interviews with the teams could shift it.

Annex: reproducibility

The sandbox builds from scratch with eleven scripts in about ten minutes and costs 2.67 credits. Every claim in this document is verifiable by a single query: build correctness — `RISK_GOV.DQ.BUILD_ASSERTIONS`, the bridge between calculations — `V_COR_RECONCILIATION`, the quality traffic light — `V_SCORECARD`, the catalog — `V_OBJECT_CATALOG`, change impact assessment — `V_CHANGE_IMPACT`, unregistered schema changes — `V_SCHEMA_DRIFT`, the agent context — `AGENT.V_AGENT_CONTEXT`.

Sandbox volume: 10 databases, 23 schemas, 59 tables, 15 views, 1 semantic view, 8,141,114 rows, 0.307 GB. Scripts and documentation are in the repository, directory `snowflake-sandbox/`.